

## A função semântica $\mathcal{S}_{\text{SOS}}$

O significado de um programa pode ser visto como uma **função parcial** de **State** para **State**.

### Função semântica $\mathcal{S}_{\text{SOS}}$

$$\mathcal{S}_{\text{SOS}} : \text{Stm} \rightarrow (\text{State} \hookrightarrow \text{State})$$

$$\mathcal{S}_{\text{SOS}}[C] s = \begin{cases} s' & \text{se } \langle C, s \rangle \Rightarrow^* s' \\ \text{undef} & \text{caso contrário} \end{cases}$$

Para qualquer programa  $C$ ,  $\mathcal{S}_{\text{SOS}}[C] \in \text{State} \hookrightarrow \text{State}$  é uma função parcial.

A boa definição de  $\mathcal{S}_{\text{SOS}}$  é uma consequência do determinismo da relação de transição.

## Semântica Natural *versus* Semântica Operacional Estrutural

## Equivalência entre semânticas

### Lema

Para quaisquer  $C, s, s'$ ,

$$\text{se } \langle C, s \rangle \rightarrow s' \text{ então } \langle C, s \rangle \Rightarrow^* s'.$$

**Prova:** por indução na estrutura da derivação de  $\langle C, s \rangle \rightarrow s'$ .

### Lema

Para quaisquer  $C, s, s'$ , e  $k$ .

$$\text{se } \langle C, s \rangle \Rightarrow^k s' \text{ então } \langle C, s \rangle \rightarrow s'.$$

**Prova:** por indução no comprimento da sequência  $\langle C, s \rangle \Rightarrow^k s'$ .

## Equivalência entre semânticas

### Teorema

Para qualquer programa  $C$  da linguagem **While**,

$$\mathcal{S}_{\text{ns}}[C] = \mathcal{S}_{\text{SOS}}[C]$$

**Prova:** Consequência direta dos dois lemas anteriores.

### Nota

Este resultado deve-se às características específicas da linguagem **While**. Com linguagens mais sofisticadas, ou na presença de situações de possíveis erros de execução, esta equivalência poderá não acontecer.

## Extensões à linguagem While

## Extensões à linguagem While

Para ilustrar o poder e as fraquezas das duas abordagens à semântica operacional, vamos considerar as seguintes pequenas **extensões à linguagem While**:

- um comando **abort**
- **execução não determinista** de dois comandos
- **execução paralela** de dois comandos

## While + abort

O comando **abort** pára a execução do programa.

- O comando **abort** é diferente de **while true do skip**.
- O comando **abort** é diferente de **skip**.

$$C ::= x := a \mid n \mid \text{skip} \mid C_1; C_2 \mid \text{if } b \text{ then } C_1 \text{ else } C_2 \\ \mid \text{while } b \text{ do } C \mid \text{abort}$$

- A semântica **big-step** é definida pelo mesmo conjunto de regras que foi definido para a linguagem **While**.
- A semântica **small-step** é definida pelo mesmo conjunto de regras que definido para a linguagem **While**.
  - ▶ As configurações  $\langle \text{abort}, s \rangle$  são bloqueadas.

## While + abort

### Semântica operacional estrutural (small-step)

- **abort** e **skip** não são semanticamente equivalentes
- **abort** e **while true do skip** não são semanticamente equivalentes

### Semântica natural (big-step)

- **abort** e **skip** não são semanticamente equivalentes
- **abort** e **while true do skip** são semanticamente equivalentes

**Porquê?**

## While + abort

- A **semântica natural** não consegue fazer a distinção entre um programa entrar em ciclo ou terminar abruptamente.
- Na **semântica operacional estrutural**,
  - ▶ o programa entrar em ciclo corresponde a uma sequência de derivação infinita
  - ▶ o programa terminar abruptamente corresponde a uma sequência de derivação finita, que termina numa configuração bloqueada.

Se considerarmos um **estado de erro** e modelarmos a terminação abrupta por uma transição para esse estado de erro (incluído nas configurações terminais), então conseguimos distinguir entre os três comandos em ambas as semânticas.

## While + or

$$C ::= x := a \mid n \mid \text{skip} \mid C_1; C_2 \mid \text{if } b \text{ then } C_1 \text{ else } C_2 \\ \mid \text{while } b \text{ do } C \mid C_1 \text{ or } C_2$$

A semântica informal do comando  $C_1 \text{ or } C_2$  é que, de modo **não determinista**, tanto pode ser executado o comando  $C_1$  como o  $C_2$ .

### Exemplo:

Após executar

$$x := 1 \text{ or } \{x := 2; x := x + 2\}$$

o valor de  $x$  tanto pode ser 1 como 4.

## While + or

### Semântica natural (*big-step*)

$$[\text{or}_{\text{sos}}^1] \quad \frac{\langle C_1, s \rangle \rightarrow s'}{\langle C_1 \text{ or } C_2, s \rangle \rightarrow s'}$$

$$[\text{or}_{\text{sos}}^2] \quad \frac{\langle C_2, s \rangle \rightarrow s'}{\langle C_1 \text{ or } C_2, s \rangle \rightarrow s'}$$

### Exercício:

Indique as árvores de derivação correspondentes às transições iniciadas nas configurações seguintes:

- 1  $\langle x := 1 \text{ or } \{x := 2; x := x + 2\}, s \rangle$
- 2  $\langle \{\text{while true do skip}\} \text{ or } \{x := 2; x := x + 2\}, s \rangle$

## While + or

### Semântica operacional estrutural (*small-step*)

$$[\text{or}_{\text{sos}}^1] \quad \langle C_1 \text{ or } C_2, s \rangle \Rightarrow \langle C_1, s \rangle$$

$$[\text{or}_{\text{sos}}^2] \quad \langle C_1 \text{ or } C_2, s \rangle \Rightarrow \langle C_2, s \rangle$$

### Exercício:

Indique as sequências de derivação correspondentes às seguintes configurações iniciais:

- 1  $\langle x := 1 \text{ or } \{x := 2; x := x + 2\}, s \rangle$
- 2  $\langle \{\text{while true do skip}\} \text{ or } \{x := 2; x := x + 2\}, s \rangle$

## While + or

- Numa **semântica natural** o não determinismo suprime a divergência.
- Numa **semântica operacional estrutural** a divergência não é suprimida pelo não determinismo.

## While + par

$$C ::= x := a \mid n \mid \text{skip} \mid C_1; C_2 \mid \text{if } b \text{ then } C_1 \text{ else } C_2 \\ \mid \text{while } b \text{ do } C \mid \text{C}_1 \text{ par } C_2$$

A semântica informal do comando  $C_1 \text{ par } C_2$  é que, ambos os comandos,  $C_1$  e  $C_2$ , executam **paralelamente de forma intercalada** (*interleaving*).

### Exemplo:

Após executar

$$x := 1 \text{ par } \{x := 2; x := x + 2\}$$

$x$  pode ser três valores diferentes: 4, 1 ou 3.

## While + par

### Semântica operacional estrutural (*small-step*)

$$[\text{par}_{\text{sos}}^1] \frac{\langle C_1, s \rangle \Rightarrow \langle C'_1, s' \rangle}{\langle C_1 \text{ par } C_2, s \rangle \Rightarrow \langle C'_1 \text{ par } C_2, s' \rangle} \quad [\text{par}_{\text{sos}}^2] \frac{\langle C_1, s \rangle \Rightarrow s'}{\langle C_1 \text{ par } C_2, s \rangle \Rightarrow \langle C_2, s' \rangle}$$

$$[\text{par}_{\text{sos}}^3] \frac{\langle C_2, s \rangle \Rightarrow \langle C'_2, s' \rangle}{\langle C_1 \text{ par } C_2, s \rangle \Rightarrow \langle C_1 \text{ par } C'_2, s' \rangle} \quad [\text{par}_{\text{sos}}^4] \frac{\langle C_2, s \rangle \Rightarrow s'}{\langle C_1 \text{ par } C_2, s \rangle \Rightarrow \langle C_1, s' \rangle}$$

### Exercício:

Indique as sequências de derivação iniciadas na configuração

$$\langle x := 1 \text{ par } \{x := 2; x := x + 2\}, s \rangle$$

## While + par

Na **semântica natural** (*big-step*), não se consegue modelar o interlaçar das execuções de  $C_1$  e  $C_2$ .

Os passos de execução são demasiado grandes (*big-step*!)

A tentativa de modelar com as seguintes regras **não funciona!!!**

$$\frac{\langle C_1, s \rangle \rightarrow s' \quad \langle C_2, s' \rangle \rightarrow s''}{\langle C_1 \text{ par } C_2, s \rangle \rightarrow s''}$$

$$\frac{\langle C_2, s \rangle \rightarrow s' \quad \langle C_1, s' \rangle \rightarrow s''}{\langle C_1 \text{ par } C_2, s \rangle \rightarrow s''}$$

## While + par

- Numa **semântica natural** a execução dos constituintes imediatos dos comandos é uma entidade atômica, o que não permite modelar o entrelaçamento das computações.
- Numa **semântica operacional estrutural** o foco é nos passos individuais de execução, o que permite facilmente modelar o entrelaçamento das computações.

## While + par + protec

### Exercício:

Considere a extensão **While+par+protect** que, para além da construção **par**, acrescenta o comando

**protect C end**

A ideia é que o comando *C* tem que ser executado como uma entidade atômica (i.e. não pode ser intercalado com outras computações).

- 1 Estenda o conjunto de regras da semântica operacional estrutural de forma a contemplar esta construção.

- 2 Indique as sequências de derivação iniciadas na configuração

$$\langle x := 1 \text{ par } \{ \text{protect } x := 2; x := x + 2 \text{ end} \}, s \rangle$$

- 3 Será possível usar uma semântica natural para exprimir a nova linguagem **While+par+protec** ?

## Erros

## Erros na avaliação de expressões

Considere agora que queremos estender a linguagem de expressões aritméticas com a operação de **divisão**.

### Como avaliar a divisão por zero ?

#### Temos diferentes opções:

- Associar à divisão por zero um valor no domínio de interpretação (considerando assim a operação válida).
- Usando semântica operacional, definir o sistema de inferência das transições de forma a que não seja possível derivar transições de expressões que envolvam divisões por zero. Teremos assim algumas **configurações bloqueadas**.
- Introduzir um valor especial de **erro**,  $\perp$ , e estender os domínios de interpretação das expressões com este valor.

$$\mathbf{Z}_{\perp} = \mathbf{Z} \cup \{\perp\} \quad \mathbf{T}_{\perp} = \mathbf{T} \cup \{\perp\}$$